OPERATING SYSTEMS

MODULE 1

Q1. Describe the different types of computer-system architectures used in modern operating systems.

1. Definition

Computer-system architecture refers to the organization and arrangement of the major hardware components of a computer system, such as processors, memory, input/output devices, and their connections.

Modern operating systems support different computer architectures to provide efficient processing, resource sharing, reliability, and performance.

2. Types of computer-system architectures

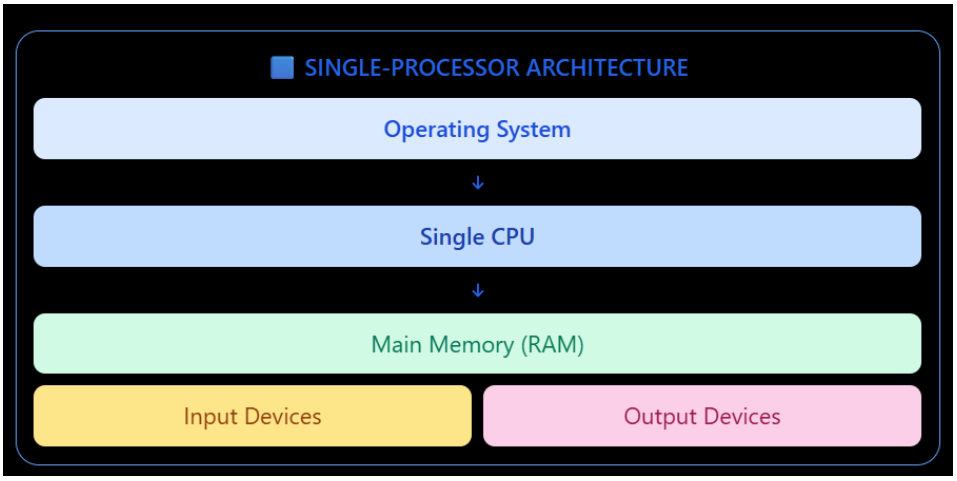
The major types are:

1. Single-processor systems
2. Multiprocessor systems
3. Multicore systems
4. Clustered systems

3. Single-processor system

A single-processor system contains one main general-purpose CPU that executes instructions and performs processing.

🟦 SINGLE-PROCESSOR ARCHITECTURE



Working:

1. The user runs a program.
2. The operating system loads the program into memory.
3. The CPU fetches and executes its instructions.
4. The OS manages CPU time and other resources.
5. Input/output devices communicate through the OS and device controllers.

Advantages:

- Simple system design.
- Easy to manage and maintain.
- Lower hardware cost.

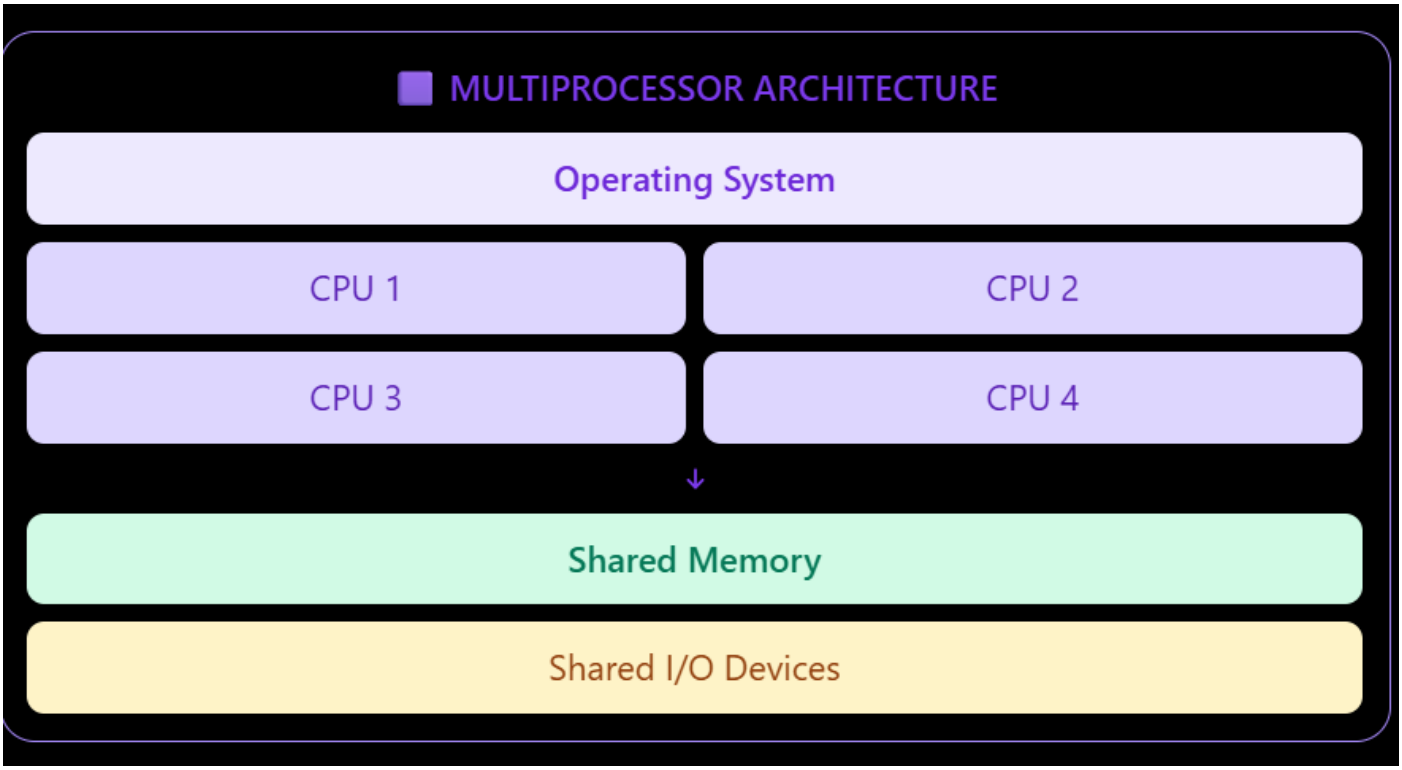
Disadvantages:

- Limited processing capacity.
- CPU can become a bottleneck.
- Less suitable for heavy parallel workloads.

4. Multiprocessor systems

A multiprocessor system contains two or more processors that share system resources such as memory and input/output devices.

It is also called a parallel system.



Types of multiprocessor systems

A. Asymmetric multiprocessing (AMP)

- Each processor is assigned a specific task.
- One processor may act as the master.
- Other processors perform assigned tasks.
- The master controls and coordinates system activities.

B. Symmetric multiprocessing (SMP)

- All processors are treated equally.
- Each processor can execute operating-system and user instructions.
- Processors share main memory and I/O resources.
- The OS schedules processes among available processors.

Advantages:

- Increased processing speed.
- Improved throughput.
- Better resource utilization.
- Greater reliability in some configurations.

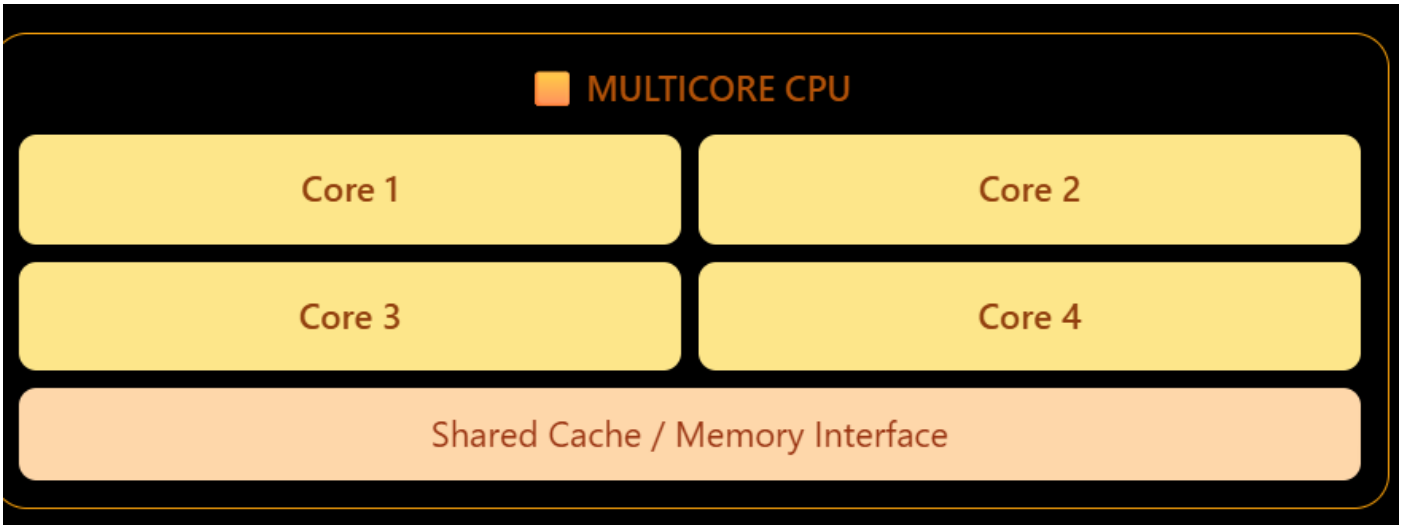
Disadvantages:

- More complex operating-system design.
- Higher hardware cost.
- Requires processor coordination and synchronization.

5. Multicore systems

A multicore system contains two or more processing cores within a single physical CPU chip.

Each core can execute instructions independently.



Important points:

1. Multiple cores are integrated into one processor chip.
2. Each core executes instructions independently.
3. Multiple tasks can execute in parallel.
4. Cores may share cache and memory resources.
5. The operating system schedules threads on available cores.

Advantages:

- Higher performance.
- Better multitasking.
- Reduced power consumption compared with many separate chips.
- Efficient parallel processing.

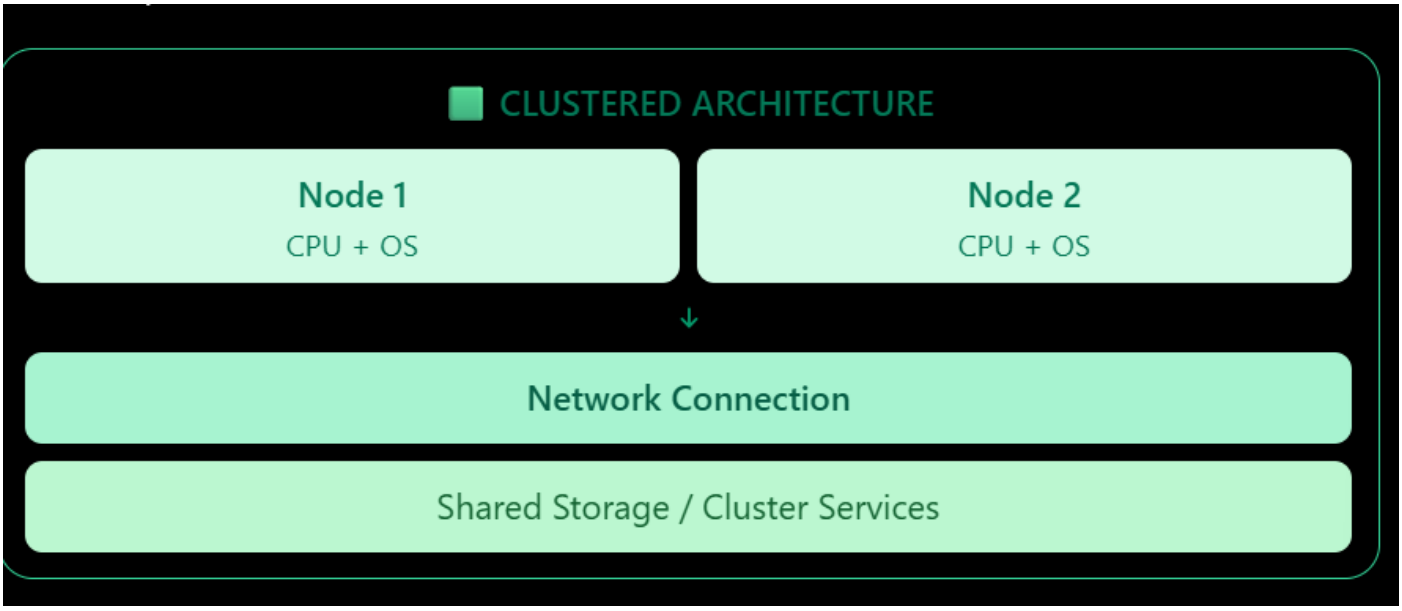
Disadvantages:

- Requires parallel software for maximum benefit.
- Shared resources may cause contention.
- Heat and power management become important.

6. Clustered systems

A clustered system consists of two or more independent computer systems connected through a network and working together.

Each computer is called a node.



Types:

- Asymmetric clustering: One node actively performs work while another remains ready for backup.
- Symmetric clustering: Multiple nodes actively perform work and monitor one another.

Advantages:

- High availability.
- Improved reliability.
- Increased processing capacity.
- Can continue working when a node fails, if configured appropriately.

Disadvantages:

- Expensive setup.
- Network configuration is required.
- More complex management.

7. Comparison table

Architecture	**Main feature**
Single processor	One main CPU
Multiprocessor	Multiple processors sharing resources
Multicore	Multiple cores on one CPU chip
Clustered	Multiple independent computers connected together

Conclusion: Modern operating systems use single-processor, multiprocessor, multicore, and clustered architectures to manage computing resources efficiently. Multiprocessor and multicore systems improve parallel processing, while clustered systems provide scalability and high availability.

Q2. Explain the major operating-system services provided to users and the system itself.

1. Definition

An operating system (OS) is system software that acts as an interface between users, application programs, and computer hardware.

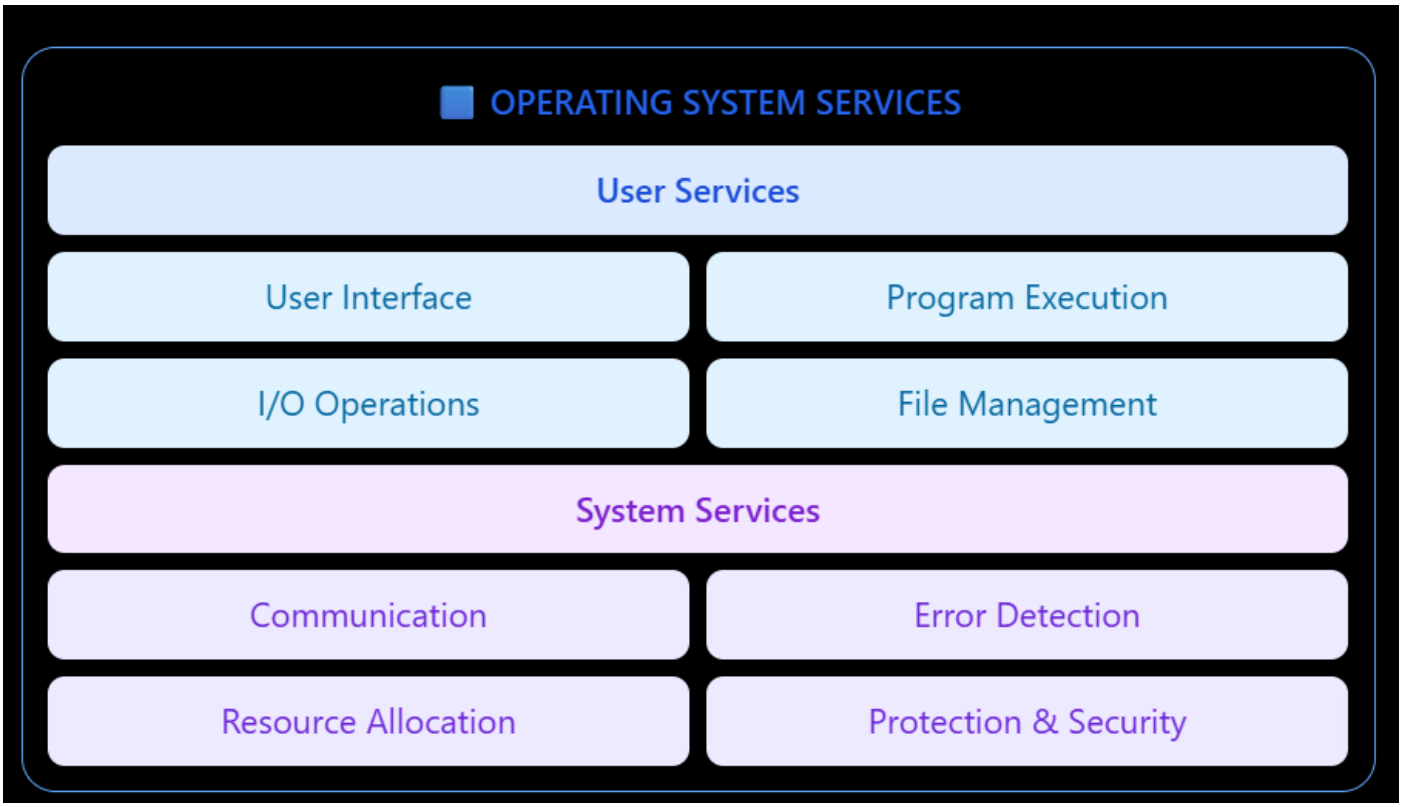
It manages hardware resources and provides services required for program execution.

Examples: Windows, Linux, macOS, Android.

2. Major operating-system services

Operating-system services can be divided into:

- Services for user convenience
- Services for efficient system operation



A. Services for user convenience

1. User interface (UI)

The user interface allows users to interact with the computer system.

Types:

- CLI: Command Line Interface; users enter commands through a terminal.
- GUI: Graphical User Interface; users interact using windows, icons, menus, and a pointer.
- Touch interface: Users interact through touch gestures.

Example: Windows desktop or Linux terminal.

2. Program execution

The OS provides the environment required to execute programs.

Steps:

1. Load the program into memory.
2. Allocate required resources.
3. Start program execution.
4. Handle program input and output.
5. Terminate the program and release resources.

Example: Opening a web browser.

3. I/O operations

The OS manages input and output devices such as keyboards, printers, disks, and monitors.

Functions:

- Controls access to devices.
- Provides device drivers.
- Performs read and write operations.
- Handles device errors.

Example: Reading keyboard input or printing a document.

4. File-system manipulation

The OS provides services for creating, reading, writing, deleting, and organizing files.

Functions:

- Create and delete files.
- Open and close files.
- Read and write data.
- Create and manage directories.

- Control file permissions.

Example: Saving a document in a folder.

5. Communication

The OS allows processes to exchange information.

Communication can occur through:

- Shared memory.
- Message passing.
- Network communication.

Example: A browser process communicating with another service.

6. Error detection

The OS detects and responds to errors in hardware and software.

Examples:

- Memory errors.
- Disk errors.
- Device failures.
- Program execution errors.

It attempts to maintain reliable system operation.

B. Services for efficient system operation

7. Resource allocation

The OS allocates resources to processes.

Resources include:

- CPU time.
- Main memory.
- Storage.
- I/O devices.

Example: CPU scheduling allocates processor time to multiple processes.

8. Accounting

The OS maintains information about resource usage.

It records:

- CPU time.
- Memory usage.
- System activity.
- User resource consumption.

This helps in monitoring and system performance analysis.

9. Protection and security

Protection controls access to system resources.

Security protects the system against unauthorized access and misuse.

Functions:

- User authentication.
- Access permissions.
- Process isolation.
- Data protection.
- Controlled access to resources.

Example: Login password and file permissions.

3. Summary table

Service	**Purpose**
User interface	User interaction
Program execution	Run applications
I/O operations	Control input/output
File management	Organize files
Communication	Exchange information
Error detection	Detect system problems
Resource allocation	Distribute resources
Accounting	Track resource usage
Protection and security	Prevent unauthorized access

Conclusion: Operating-system services make computers convenient to use and ensure efficient, secure, and reliable resource management.

Q3. Explain the abstract view of a computer system with a neat diagram and show how its major components interact with the operating system.

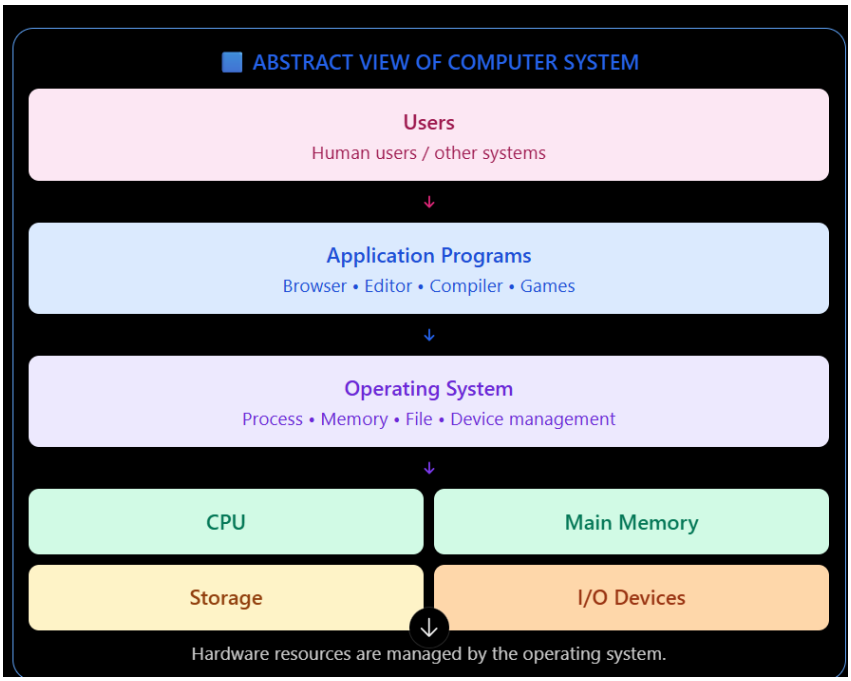
1. Definition

An abstract view of a computer system represents the computer as a collection of major components and shows how they interact without describing every internal hardware detail.

A computer system is commonly divided into four major components:

1. Hardware
2. Operating system
3. Application programs
4. Users

2. Neat diagram



3. Major components

A. Hardware

Hardware refers to the physical components of a computer system.

Examples:

- CPU.
- Main memory.
- Hard disk or SSD.
- Keyboard.

- Mouse.
- Monitor.
- Network devices.

Functions:

1. CPU executes instructions.
2. Memory stores programs and data currently in use.
3. Storage keeps data permanently.
4. I/O devices provide communication with the external environment.

B. Operating system

The operating system is the system software that controls hardware and provides services to application programs.

Functions:

- Process management.
- Memory management.
- File management.
- Device management.
- Security and protection.
- Resource allocation.

It acts as an intermediary between applications and hardware.

C. Application programs

Application programs perform specific tasks for users.

Examples:

- Web browsers.
- Word processors.
- Compilers.
- Games.
- Database applications.

They use OS services to access hardware resources.

D. Users

Users interact with the system through application programs and user interfaces.

Users may be:

- Individuals.
- Organizations.
- Other computer systems.

4. How components interact

Consider opening a web browser:

1. The user clicks the browser icon.
2. The application requests program execution from the OS.
3. The OS creates a process and allocates memory.
4. The CPU executes browser instructions.
5. The browser requests network access through OS services.
6. The OS communicates with the network device.
7. Data is received and placed in memory.
8. The browser displays the result on the monitor.

5. Important points

- Hardware provides physical resources.
- The OS manages and coordinates hardware.
- Applications use OS services.
- Users interact with applications.
- The OS ensures efficient and controlled resource usage.

Conclusion: The abstract view shows that the operating system connects application programs and users with hardware. It manages resources and provides a convenient environment for program execution.

Q4. Demonstrate the working of process scheduling using a suitable example and explain how the scheduler selects a process for execution.

1. Definition

Process scheduling is the activity of selecting a process from the ready queue and allocating the CPU to it for execution.

A process is a program in execution.

A process scheduler is an operating-system component that selects a process from the available processes according to a scheduling algorithm.

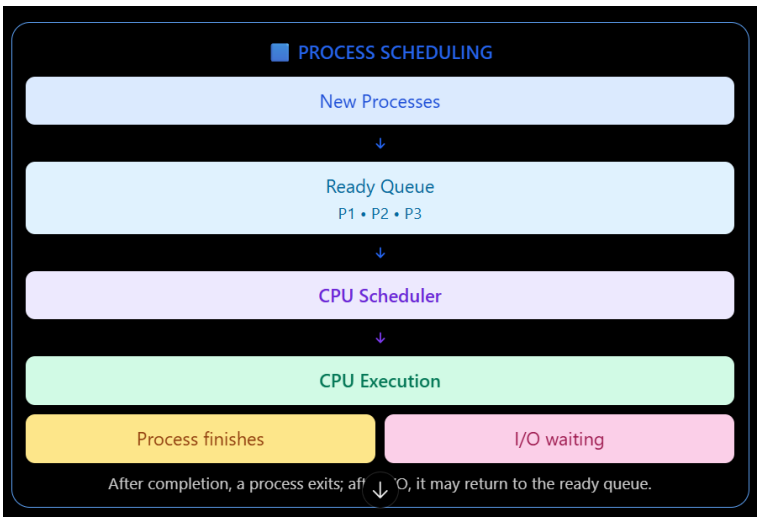
2. Objectives of process scheduling

1. Maximize CPU utilization.
2. Increase system throughput.
3. Minimize waiting time.
4. Minimize turnaround time.
5. Provide quick response time.
6. Ensure fairness among processes.

3. Types of scheduling

- Preemptive scheduling: The OS can interrupt a running process and allocate the CPU to another process.
- Non-preemptive scheduling: A process continues until it finishes or voluntarily waits for an event.

5. Process scheduling diagram



5. Example: First Come First Served (FCFS)

FCFS is a non-preemptive scheduling algorithm in which the process that arrives first gets the CPU first.

Consider three processes:

Process	Arrival time	Burst time
P1	0	5
P2	1	3
P3	2	4

Arrival time: Time when a process enters the ready queue.

Burst time: CPU time required by a process to complete its execution.

Step 1: Arrange processes

Processes arrive in the order:

P1 → P2 → P3

Step 2: Select the first process

P1 arrives at time 0.

Therefore, P1 is selected first.

P1 executes from time 0 to time 5.

Step 3: Select the next process

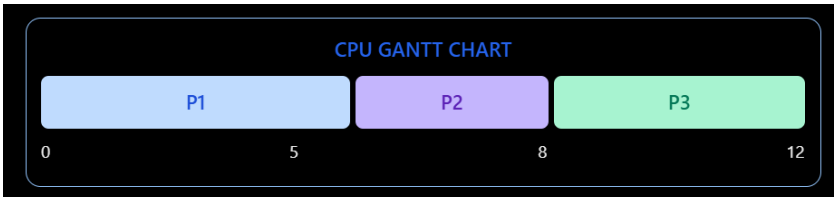
P2 has already arrived and is waiting.

P2 executes from time 5 to time 8.

Step 4: Select the remaining process

P3 executes from time 8 to time 12.

Step 5: Draw the Gantt chart



Step 6: Calculate scheduling times

Completion time (CT): Time at which a process finishes.

Turnaround time (TAT):

\[TAT=CT-AT \]

Waiting time (WT):

$$ WT=TAT-BT $$

Process	CT	TAT = CT − AT	WT = TAT − BT
P1	5	5 − 0 = 5	5 − 5 = 0
P2	8	8 − 1 = 7	7 − 3 = 4
P3	12	12 − 2 = 10	10 − 4 = 6

Average waiting time:

$$ \frac{0+4+6}{3}=\boxed{3.33} $$

Average turnaround time:

$$ \frac{5+7+10}{3}=\boxed{7.33} $$

7. How the scheduler selects a process

1. Processes enter the ready queue.
2. The scheduler checks the scheduling policy.
3. It selects an eligible process.
4. The dispatcher gives the CPU to that process.
5. The process executes.
6. When it finishes, blocks, or is preempted, scheduling occurs again.

Conclusion: Process scheduling enables the OS to share the CPU among multiple processes. FCFS selects processes in arrival order, while other algorithms may select based on burst time, priority, or time quantum.

Q5. Analyze the functions of an operating system and relate them to computer-system architecture and operating-system operations.

1. Definition

An operating system is system software that manages computer hardware and software resources and provides services to application programs.

It acts as an intermediary between the user and the computer hardware.

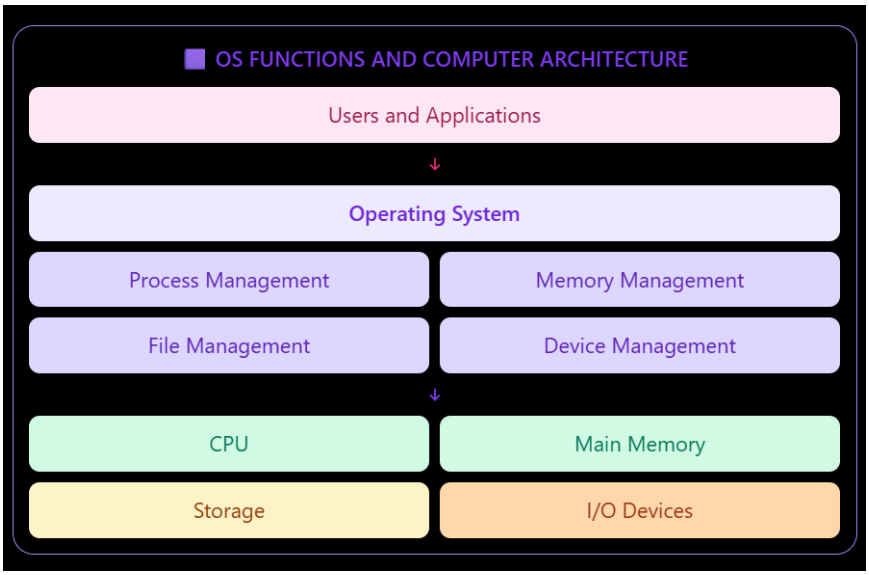
2. Functions of an operating system

The major functions are:

1. Process management.
2. Memory management.

3. File-system management.
4. I/O device management.
5. Secondary-storage management.
6. Protection and security.
7. Resource allocation.
8. Error detection.
9. Networking and communication.
10. User-interface management.

3. OS and computer architecture diagram



OS FUNCTIONS AND COMPUTER ARCHITECTURE

4. Detailed functions

1. Process management

A process is a program currently being executed.

The OS:

- Creates and terminates processes.
- Allocates CPU time.
- Suspends and resumes processes.
- Performs process scheduling.
- Supports synchronization and communication.

Architecture relation: The CPU executes processes, and the OS decides which process receives CPU time.

2. Memory management

Memory management is the process of controlling and organizing main memory.

The OS:

- Tracks memory usage.
- Allocates memory to processes.
- Releases memory after use.
- Protects process memory.
- Supports virtual memory.

Architecture relation: Main memory stores the instructions and data currently required by the CPU.

3. File-system management

A file system organizes and stores files on storage devices.

The OS:

- Creates and deletes files.
- Creates directories.
- Reads and writes files.
- Manages permissions.
- Maintains file information.

Architecture relation: Storage devices provide persistent data, and the OS manages how applications access it.

4. I/O device management

The OS manages input and output devices.

Functions:

- Device allocation.
- Device-driver management.
- Read/write operations.
- Interrupt handling.
- Device error handling.

Architecture relation: Device controllers and drivers connect the OS to hardware devices.

5. Secondary-storage management

Secondary storage includes SSDs and hard disks.

The OS:

- Manages free space.
- Allocates storage blocks.
- Organizes stored data.
- Schedules disk operations.

6. Security and protection

The OS controls access to system resources.

It provides:

- Authentication.
- Authorization.
- Access permissions.
- Process isolation.
- Protection of files and memory.

7. Resource allocation

The OS distributes resources among processes.

Examples:

- CPU scheduling.
- Memory allocation.
- Device allocation.
- Storage allocation.

8. Error detection

The OS detects and handles errors such as:

- CPU exceptions.
- Memory faults.
- Disk errors.
- Device failures.

9. Communication

The OS supports communication between processes and computers.

Methods:

- Shared memory.
- Message passing.
- Network communication.

10. User-interface management

The OS provides interfaces through which users operate the computer.

Examples:

- Command-line interface.
- Graphical user interface.
- Touch interface.

5. Relationship with OS operations

Operating-system operations are the activities through which the OS controls hardware and manages execution.

Important operations include:

1. Interrupt handling: Responds to hardware and software interrupts.
2. Dual-mode operation: Distinguishes user mode from kernel mode.
3. System calls: Allow user programs to request OS services.
4. Timer operation: Prevents a process from using the CPU indefinitely.
5. Resource management: Coordinates CPU, memory, and devices.

6. Summary table

OS function	**Related hardware**
Process management	CPU
Memory management	RAM
File management	SSD / HDD
Device management	I/O controllers
Storage management	Secondary storage
Security	CPU, memory, storage

Communication	Network devices

Conclusion: The operating system connects computer-system architecture with application execution. It manages CPU, memory, storage, and I/O devices while providing efficient, secure, and reliable system operation.

Q6. Apply the major approaches used for organizing operating-system components and illustrate the operating-system structure with a suitable diagram.

1. Definition

Operating-system structure refers to the way OS components are organized and connected to provide system services.

The structure determines how the kernel, system programs, device drivers, and user applications communicate.

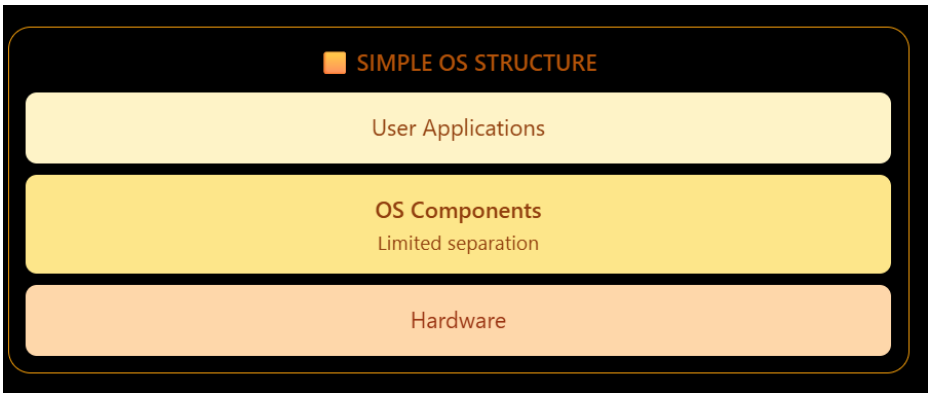
2. Major approaches to OS structure

1. Simple structure.
2. Monolithic structure.
3. Layered structure.
4. Microkernel structure.
5. Modular structure.
6. Hybrid structure.

3. Simple structure

A simple-structure OS has a small number of components with little separation between them.

Example: Early MS-DOS.



Characteristics:

- Simple design.
- Components are not strongly separated.
- Easy to develop for small systems.
- Limited protection between components.

Advantages:

- Easy to understand.
- Small implementation overhead.
- Suitable for simple systems.

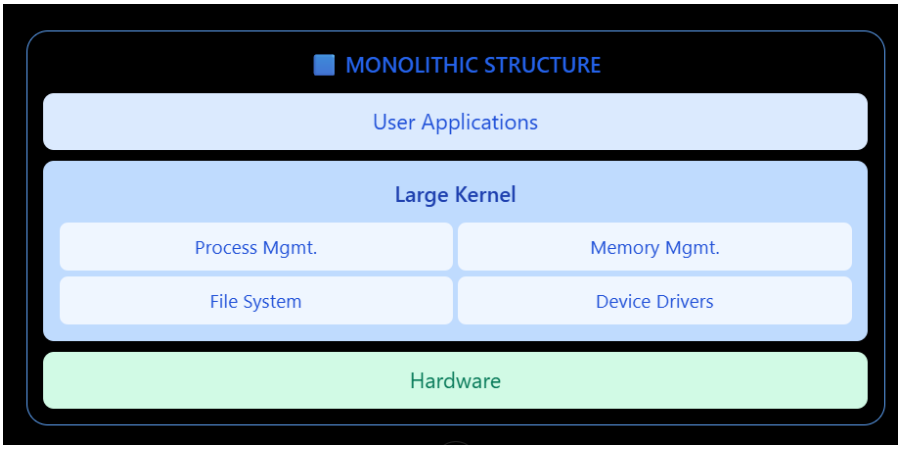
Disadvantages:

- Difficult to maintain.
- Poor modularity.
- A fault can affect other components.
- Security and reliability are limited.

4. Monolithic structure

A monolithic operating system places most OS services inside one large kernel running in kernel mode.

Examples: Traditional UNIX and Linux's monolithic kernel design.



Working:

1. User applications request OS services.
2. System calls enter kernel mode.
3. Kernel components perform process, memory, file, and device operations.
4. Results are returned to the application.

Advantages:

- High performance.
- Direct communication between kernel components.
- Efficient system-service execution.

Disadvantages:

- Large kernel is difficult to maintain.
- Kernel bugs can affect the entire system.
- Debugging can be complex.

5. Layered structure

A layered OS is divided into different layers, where each layer performs a specific function.

Each layer uses services provided by the layer below it.

🟪 LAYERED OS STRUCTURE

Layer 5: User Programs

Layer 4: System Services

Layer 3: File & I/O Management

Layer 2: Memory & Process Management

Layer 1: Hardware

Higher layers use services provided by lower layers.

Advantages:

- Modular design.
- Easier debugging.
- Clear separation of responsibilities.
- Easier maintenance.

Disadvantages:

- Defining suitable layers is difficult.
- Requests may pass through multiple layers.
- Extra layer transitions can reduce performance.

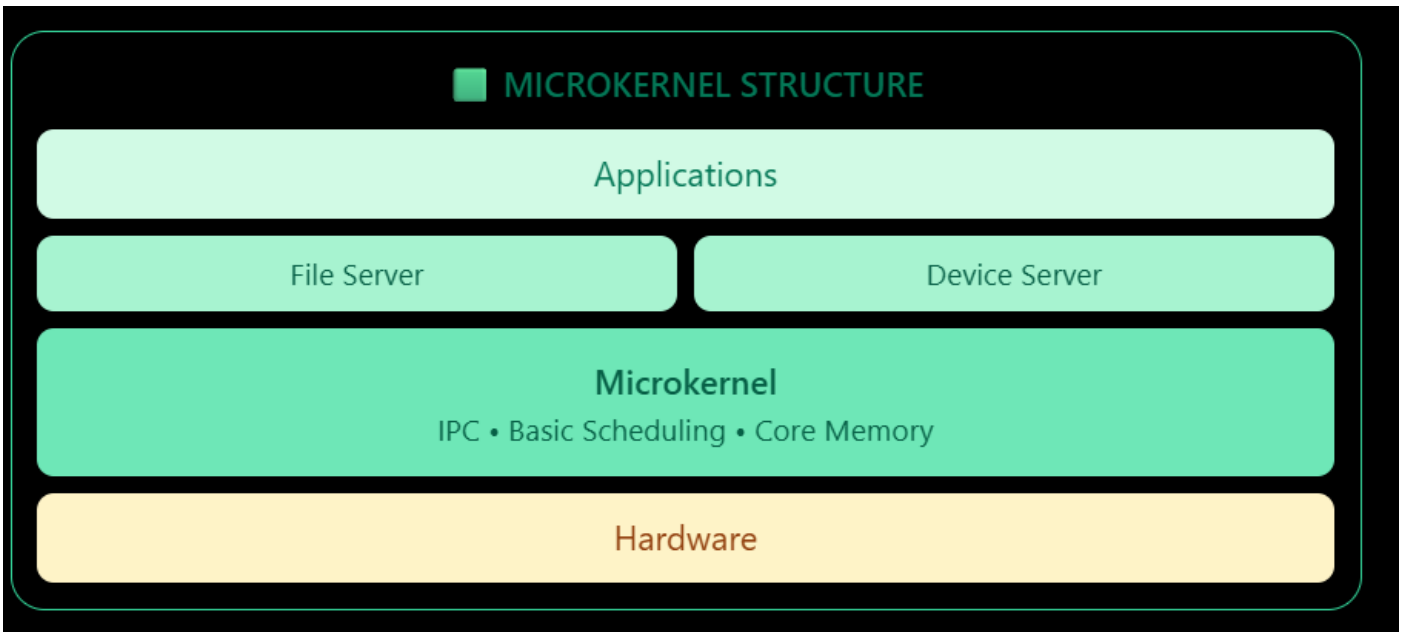
6. Microkernel structure

A microkernel keeps only essential OS functions inside the kernel.

Other services run in user space.

Essential functions may include:

- Basic scheduling.
- Inter-process communication.
- Low-level memory management.



Advantages:

- Improved isolation.
- Better reliability.
- Easier to extend some services.
- Smaller kernel.

Disadvantages:

- Communication between servers can add overhead.
- System design is complex.
- Performance may be affected by message passing.

Examples: MINIX 3 and QNX.

7. Modular structure

A modular OS uses a core kernel and separate modules that can be loaded or unloaded.

Modules may provide:

- Device drivers.
- File systems.
- Network services.

💗 MODULAR OS

Applications

Core Kernel

Driver

File System

Network

Hardware

Advantages:

- Flexible design.
- Easier to add features.
- Better organization.
- Modules can be loaded when required.

Disadvantages:

- Module dependencies must be managed.
- Faulty kernel modules can affect the system.

Example: Linux supports loadable kernel modules.

8. Hybrid structure

A hybrid OS combines features of multiple OS structures, such as monolithic, layered, and microkernel approaches.

Examples: Windows and macOS use hybrid designs.

Advantages:

- Combines performance and modularity.
- Flexible system design.
- Supports different OS services efficiently.

Disadvantages:

- Complex architecture.
- Requires careful integration.
- Debugging can be difficult.

9. Comparison table

Structure	Main characteristic
Simple	Minimal separation
Monolithic	Most services inside kernel
Layered	Organized into hierarchical layers
Microkernel	Minimal kernel, services outside
Modular	Kernel supports separate modules
Hybrid	Combines multiple approaches

Conclusion: Operating systems use different structures depending on their performance, reliability, and maintenance requirements. Modern systems commonly combine structural approaches to achieve efficient and flexible operation.

Q7. Differentiate between user and operating-system interfaces and analyze how each approach enables users to interact with the operating system.

1. Definition

A user interface (UI) is the method through which a user communicates with a computer system.

An operating-system interface is the method through which users or programs access operating-system services.

The user interface is intended for human interaction, while system interfaces provide controlled access to OS functions.

2. Types of user interfaces

A. Command-line interface (CLI)

A CLI allows users to interact with the operating system by typing commands.

Example: Linux terminal, Windows Command Prompt.

🟦 COMMAND-LINE INTERFACE

C:\> dir
C:\> cd Documents
C:\> mkdir OS

Users enter commands and receive text output.

Advantages:

- Fast for experienced users.
- Requires fewer system resources.
- Supports automation and scripts.
- Provides precise control.

Disadvantages:

- Commands must be learned.
- Typing errors may occur.
- Less convenient for beginners.

B. Graphical user interface (GUI)

A GUI allows users to interact with the OS through graphical elements such as windows, icons, menus, and buttons.

Example: Windows desktop.

🟪 GRAPHICAL USER INTERFACE

Desktop

Folder

File

Settings

Users click graphical elements to perform operations.

Advantages:

- Easy to learn.
- Visual interaction.
- Convenient for everyday use.
- Supports mouse and touch.

Disadvantages:

- Requires more system resources.

- Some operations are slower than CLI.
- Less convenient for command automation.

C. Touch interface

A touch interface allows users to interact with the system using fingers or a stylus.

Examples: Android and iOS.

Operations include:

- Tap.
- Swipe.
- Pinch.
- Long press.

Advantages:

- Natural interaction.
- Convenient on mobile devices.
- Easy for common tasks.

Disadvantages:

- Less precise for some tasks.
- Screen size limits interaction.
- Typing can be less convenient.

3. Operating-system program interface

Programs use OS interfaces to request services.

A. System calls

A system call is a mechanism through which a user program requests a service from the OS kernel.

Examples:

- open() – open a file.
- read() – read data.
- write() – write data.
- fork() – create a process in UNIX-like systems.

B. Application programming interface (API)

An API is a set of functions and rules that applications use to request services from software or an operating system.

Examples:

- POSIX API.
- Windows API.

An API may internally use system calls.

4. Comparison table

Feature	CLI	GUI	Touch interface
Interaction	Typed commands	Icons and windows	Touch gestures
Learning	Requires commands	Easier for beginners	Intuitive
Speed	Fast for experts	Visual and convenient	Fast for simple tasks
Resources	Usually lower	Usually higher	Depends on device
Automation	Excellent	Limited	Limited
Example	Linux terminal	Windows desktop	Android

5. How interfaces enable interaction

1. User provides input through CLI, GUI, or touch.
2. The interface interprets the request.
3. The application or interface requests OS services.
4. The OS checks permissions and resources.
5. The OS performs the requested operation.
6. The result is displayed to the user.

Conclusion: User interfaces provide convenient interaction, while operating-system interfaces provide controlled access to system services. CLI, GUI, and touch interfaces serve different user needs.

Q8. Analyze the role of system calls in providing an interface between user programs and the operating system, with suitable examples.

1. Definition

A system call is a programmatic mechanism through which a user-level program requests a service from the operating-system kernel.

System calls provide a controlled interface between application programs and hardware-managed resources.

Examples include file access, process creation, memory allocation, and device operations.

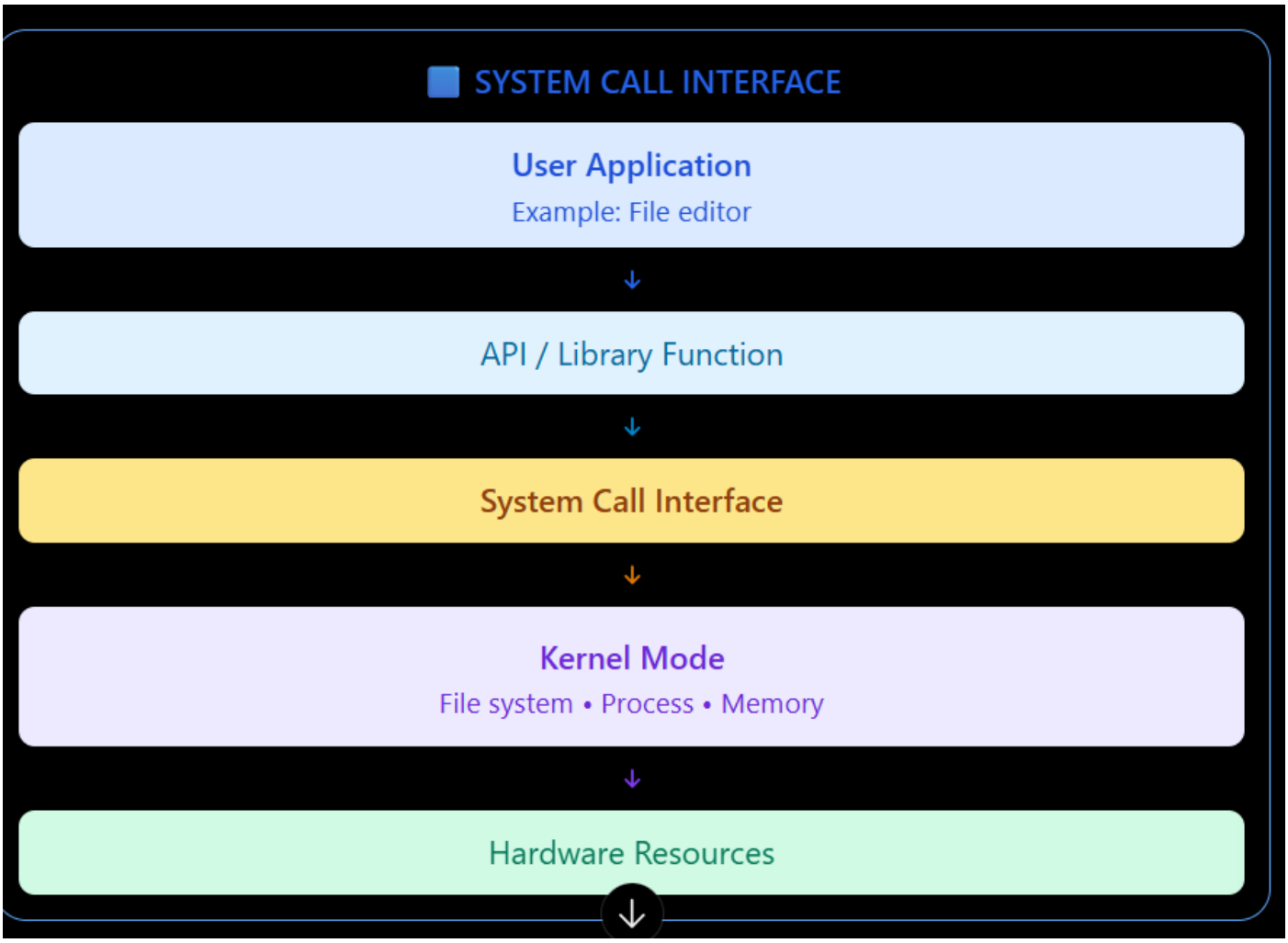
2. Why system calls are required

User programs normally execute in user mode, where access to privileged hardware operations is restricted.

The operating-system kernel executes in kernel mode, where it can perform privileged operations.

System calls allow user programs to request these operations safely.

3. System-call architecture



4. Working of a system call

Consider a program that reads a file.

Step 1: The user runs a file-reading application.

Step 2: The application calls a library function such as read().

Step 3: The system-call interface transfers control to the kernel.

Step 4: The kernel checks the file descriptor, permissions, and requested operation.

Step 5: The kernel performs the file-reading operation.

Step 6: The result is returned to the user program.

Step 7: The program continues execution in user mode.

5. Example of a system call

In a UNIX-like system, a program can use write() to request output.

```
#include <unistd.h>


int main(void)
{
   write(1, "Hello OS\n", 9);
   return 0;
}
```

Explanation:

- write() requests the OS to write data.
- 1 represents standard output.
- "Hello OS\n" is the message.
- 9 is the number of bytes requested.
- The OS performs the output operation.

6. Role of system calls

1. Process management: Create, execute, and terminate processes.
2. File management: Create, open, read, write, and close files.
3. Device management: Access hardware devices.
4. Memory management: Request and release memory.
5. Communication: Exchange information between processes.
6. Protection: Check permissions and restrict unauthorized operations.

7. Advantages

- Provides controlled access to hardware.

- Protects kernel resources.
- Simplifies application development.
- Supports standard OS operations.
- Improves security and reliability.

8. Limitations

- System-call transitions have overhead.
- Incorrect requests may fail.
- Applications depend on available OS interfaces.

Conclusion: System calls form the essential interface between user programs and the OS kernel. They provide secure, controlled, and standardized access to system resources.

Q9. Identify the different types of system calls and relate each type to appropriate operating-system operations.

1. Definition

A system call is a request made by a user program to the operating-system kernel to perform a specific operation.

System calls are classified according to the services they provide.

2. Major types of system calls

1. Process control.
2. File management.
3. Device management.
4. Information maintenance.
5. Communication.
6. Protection.

3. System-call classification diagram


TYPES OF SYSTEM CALLS

System Calls

Process Control

File Management

Device Management

Information

Communication

Protection

4. Detailed explanation

1. Process-control system calls

These system calls manage process execution.

Operations:

- Create a process.
- Terminate a process.
- Load and execute a program.
- Wait for a process.
- Allocate and release memory.

Examples in UNIX-like systems:

- fork() – create a child process.
- exec() – replace the current process image with a program.
- exit() – terminate a process.
- wait() – wait for a child process.

2. File-management system calls

These system calls manage files and directories.

Operations:

- Create files.
- Delete files.
- Open and close files.
- Read and write data.
- Change file attributes.

Examples:

- open()
- read()
- write()

- close()
- unlink()

3. Device-management system calls

These system calls provide access to devices.

Operations:

- Request device access.
- Release devices.
- Read from devices.
- Write to devices.
- Obtain device information.

Example: Reading input from a device or writing output to a device.

The exact calls depend on the operating system and device interface.

4. Information-maintenance system calls

These system calls obtain or modify information about the system.

Operations:

- Get system time.
- Get process information.
- Obtain system attributes.
- Read or update selected system data.

Examples:

- getpid() – obtain process ID.
- time() – obtain time information.
- uname() – obtain system information on UNIX-like systems.

5. Communication system calls

These system calls allow processes to exchange data.

Methods:

- Shared memory.
- Message passing.
- Pipes.

- Network communication.

Examples:

- pipe() – create a pipe.
- shmget() – obtain a shared-memory segment in System V IPC.
- send() and recv() – communicate through supported socket interfaces.

6. Protection system calls

These system calls control access to resources.

Operations:

- Set file permissions.
- Check access rights.
- Change ownership.
- Control user identity or privileges.

Examples:

- chmod() – change file permissions.
- chown() – change file ownership.
- access() – check access permissions in applicable UNIX-like systems.

5. Comparison table

Type	**Purpose**	**Example**
Process control	Manage processes	fork(), exit()
File management	Manage files	open(), read()
Device management	Access devices	Device-specific operations
Information maintenance	Obtain system information	getpid(), time()
Communication	Exchange data	pipe(), send()
Protection	Control access	chmod(), chown()

Conclusion: System calls are classified according to the services they provide. They enable applications to manage processes, files, devices, information, communication, and security through the OS kernel.

Q10. Compare and analyze the major categories of system programs and their role in providing a convenient environment for program development and execution.

1. Definition

System programs are utility programs provided with an operating system that make it easier to develop, execute, manage, and maintain application programs.

They operate above the kernel and use OS services to perform common tasks.

Examples include compilers, command interpreters, file utilities, and system-monitoring tools.

2. Major categories of system programs

1. File management.
2. Status information.
3. File modification.
4. Programming-language support.
5. Program loading and execution.
6. Communication.
7. Background services.

3. System-program structure

🟩 SYSTEM PROGRAMS

User

System Programs / Utilities

Compiler

File Utilities

Loader

Communication

Operating System

Hardware

4. Detailed categories

1. File-management programs

These programs create, delete, copy, move, rename, and organize files.

Examples:

- File explorer.
- cp – copy files in UNIX-like systems.
- mv – move or rename files.
- rm – remove files.

Role: Helps users organize files and directories.

2. Status-information programs

These programs display system information.

They provide:

- Date and time.
- Memory usage.
- CPU utilization.
- Disk usage.
- Process information.

Examples:

- Task Manager.
- top.
- ps.
- df.

Role: Helps users monitor system performance and resource usage.

3. File-modification programs

These programs modify file contents.

Examples:

- Text editors.
- Word processors.
- Search-and-replace utilities.
- File-format tools.

Role: Supports writing, editing, and maintaining program source files and documents.

4. Programming-language support

These programs help users develop software.

They include:

- Compilers.
- Assemblers.
- Interpreters.
- Debuggers.
- Libraries.

Examples:

- GCC compiler.
- Java compiler.
- Python interpreter.

Role: Converts source code into executable instructions or executes programs through an interpreter.

5. Program loading and execution

These programs prepare applications for execution.

They include:

- Loaders.
- Linkers.
- Runtime environments.
- Debuggers.

Role:

1. Load program instructions.
2. Resolve required components.
3. Prepare execution.
4. Start the program.
5. Support debugging and runtime operations.

6. Communication programs

These programs enable data exchange between users and computer systems.

Examples:

- Email applications.

- Web browsers.
- Remote-login tools.
- File-transfer utilities.

Role: Supports network communication and information sharing.

7. Background services

Background services are programs that perform tasks without direct user interaction.

Examples:

- Printing service.
- Network service.
- Update service.
- Logging service.

Role: Provide ongoing system functions and support applications.

5. Program development and execution flow

🟧 PROGRAM DEVELOPMENT FLOW

Source Code

Compiler / Interpreter

Object Code / Executable

Loader

Program Execution

6. Comparison table

Category	**Main function**	**Example**
File management	Organize files	File Explorer
Status information	Monitor system	Task Manager
File modification	Edit contents	Text editor
Language support	Develop programs	Compiler
Loading and execution	Run programs	Loader
Communication	Exchange information	Browser
Background services	Perform system tasks	Print service

7. Advantages of system programs

- Simplify program development.
- Improve user productivity.
- Provide convenient file operations.
- Support debugging.
- Help monitor system performance.
- Make application execution easier.

Conclusion: System programs provide a convenient environment for users and developers. They simplify file management, programming, execution, communication, and system monitoring.

Q11. Identify the process concept and explain how process states and process operations are related during process execution.

1. Definition of process

A process is a program in execution. It includes program instructions, data, CPU registers, stack, and other information required for execution.

A program is passive, while a process is active.

Example: A saved browser application is a program; when opened and running, it becomes a process.

2. Process components

A process generally contains:

1. Text section: Executable instructions.
2. Data section: Global and static variables.
3. Heap: Dynamically allocated memory.
4. Stack: Function calls, local variables, and return information.
5. Process Control Block (PCB): Information maintained by the OS about the process.

3. Process memory layout

🟦 PROCESS MEMORY LAYOUT

Text / Code

Program instructions

Data

Global and static variables

Heap

Dynamic memory

Stack

Function calls and local variables

4. Process Control Block (PCB)

A Process Control Block (PCB) is a data structure maintained by the OS that stores information about a process.

The PCB contains:

- Process ID (PID).
- Process state.
- Program counter.
- CPU registers.
- CPU scheduling information.
- Memory-management information.
- I/O status information.
- Accounting information.

The OS uses the PCB to manage and resume processes.

5. Process states

A process moves through different states during execution.

New: The process is being created.

Ready: The process is waiting for CPU allocation.

Running: The process is currently executing.

Waiting / Blocked: The process is waiting for an event, such as I/O completion.

Terminated: The process has completed execution.

6. Process-state diagram

PROCESS STATE TRANSITIONS

New

Ready

Scheduler dispatch

Running

Waiting

I/O request

Terminated

Exit

Waiting → Ready when event completes

Running → Ready can occur when a process is preempted.

7. Process operations

A. Process creation

The OS creates a process when a program is started or another process requests creation.

Steps:

1. Assign a process ID.
2. Create and initialize its PCB.
3. Allocate required resources.
4. Prepare program memory.
5. Place the process in the ready queue.

B. Process scheduling

The scheduler selects a process from the ready queue.

The selected process moves from ready to running.

C. Process blocking

A running process may request I/O or wait for an event.

It moves from running to waiting.

D. Process wake-up

When the required event completes, the process moves from waiting to ready.

E. Process termination

When execution finishes, the OS releases resources and marks the process terminated.

8. Example

Consider a process that reads a file:

1. Process is created.
2. It enters the ready state.
3. CPU scheduler selects it.
4. It executes.
5. It requests file input and enters waiting.
6. File input completes.
7. It returns to ready.
8. It executes again.
9. It finishes and enters terminated state.

Conclusion: Process states describe the current condition of a process. Process operations cause state transitions, while the PCB stores information needed by the OS to manage execution.

Q12. Analyze process scheduling and compare the roles of the long-term, short-term, and medium-term schedulers.

1. Definition

Process scheduling is the OS activity of selecting processes for execution and managing their movement between scheduling queues.

Scheduling helps the OS share the CPU among multiple processes.

2. Types of schedulers

1. Long-term scheduler.
2. Short-term scheduler.
3. Medium-term scheduler.

3. Scheduling diagram

🟦 PROCESS SCHEDULERS

Job Pool

Long-Term Scheduler

Ready Queue in Main Memory

Short-Term Scheduler

CPU

Medium-Term Scheduler: Suspend / Resume

4. Long-term scheduler

The long-term scheduler, also called the job scheduler, selects processes from the job pool and admits them into main memory.

Functions:

1. Selects jobs for execution.
2. Controls admission into the ready queue.
3. Determines the degree of multiprogramming.
4. Helps maintain a suitable mix of CPU-bound and I/O-bound processes.

Frequency: Runs relatively infrequently.

Importance: Controls how many processes enter the system.

5. Short-term scheduler

The short-term scheduler, also called the CPU scheduler, selects a process from the ready queue and allocates the CPU to it.

Functions:

1. Selects the next process.
2. Allocates CPU time.
3. Supports preemptive or non-preemptive scheduling.
4. Works with the dispatcher to switch execution.

Frequency: Runs very frequently.

Importance: Must operate quickly because it directly affects CPU utilization and response time.

6. Medium-term scheduler

The medium-term scheduler temporarily removes processes from main memory and later brings them back.

This process is called swapping or suspension/resumption.

Functions:

1. Suspends selected processes.
2. Reduces memory pressure.
3. Adjusts the degree of multiprogramming.

4. Resumes processes when resources are available.

Frequency: Runs when memory or workload conditions require it.

7. Comparison table

Feature	Long-term	Short-term	Medium-term
Other name	Job scheduler	CPU scheduler	Swapper
Main task	Admit jobs	Select CPU process	Suspend/resume processes
Selects from	Job pool	Ready queue	Suspended processes
Frequency	Infrequent	Very frequent	Occasional
Controls	Admission	CPU allocation	Memory load
Main objective	Job mix	CPU efficiency	Manage multiprogramming

8. Example

Suppose 10 programs are waiting to enter the system:

1. The long-term scheduler admits selected programs into memory.
2. Admitted processes enter the ready queue.
3. The short-term scheduler selects one process for CPU execution.
4. A process may block while waiting for I/O.
5. If memory becomes crowded, the medium-term scheduler may suspend a process.
6. When memory becomes available, the suspended process can resume.

Conclusion: The three schedulers perform different but related roles. The long-term scheduler controls admission, the short-term scheduler controls CPU execution, and the medium-term scheduler manages process suspension and memory pressure.

Q13. Make use of operations performed on processes, including process creation, process termination, and parent-child relationships.

1. Definition

Process operations are the activities performed by an operating system to create, execute, manage, and terminate processes.

The major operations include:

- Process creation.
- Process execution.

- Parent-child process management.
- Process waiting.
- Process termination.

2. Process creation

Process creation is the operation of starting a new process.

A process that creates another process is called the parent process.

The newly created process is called the child process.

3. Parent-child relationship diagram

🟪 PARENT-CHILD PROCESS TREE

Parent Process P0

Child P1

Child P2

Grandchild P3

Grandchild P4

4. Steps involved in process creation

1. A running process requests a new process.
2. The OS assigns a unique process ID.
3. The OS creates a PCB.
4. Memory and required resources are allocated.
5. The child process is initialized.
6. The child is placed in the appropriate scheduling queue.
7. The child becomes eligible for execution.

5. Process creation in UNIX-like systems

Common system calls:

- fork() – creates a child process.
- exec() – loads and executes a new program in the current process.
- wait() – allows a parent to wait for a child.
- exit() – terminates a process.

Example:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
   pid_t pid = fork();

   if (pid == 0) {
      printf("Child process\n");
   }
   else if (pid > 0) {
      wait(NULL);
      printf("Parent process\n");
   }
   else {
      printf("Process creation failed\n");
   }

   return 0;
}
```

Explanation:

1. fork() creates a child process.
2. The child receives a return value of 0.
3. The parent receives the child's process ID.
4. The parent waits for the child.
5. The child prints its message.
6. The parent prints its message after the child finishes.

6. Process termination

Process termination is the operation of ending a process and releasing its resources.

Reasons for termination:

- Normal completion.
- Error during execution.
- User request.
- Operating-system intervention.
- Termination by another authorized process.

7. Steps in process termination

1. The process completes or receives a termination request.
2. The OS records its exit status.
3. Resources such as memory and open files are released.
4. The process is removed from active scheduling.
5. The parent may collect the child's exit status.

8. Parent-child relationship

A parent may create multiple children.

The parent and child may:

- Execute concurrently.
- Share selected resources.
- Communicate through IPC.
- Wait for one another.
- Terminate independently, depending on OS behavior.

9. Important terms

Orphan process: A child process whose parent terminates while the child is still running. The OS may reassign it to a designated system process.

Zombie process: In UNIX-like systems, a terminated child whose exit status has not yet been collected by its parent. It retains a small process-table entry until reaped.

Conclusion: Process creation and termination are essential OS operations. Parent-child relationships allow processes to organize tasks, execute concurrently, communicate, and manage completion.

Q14. Utilize the mechanisms of inter-process communication and compare shared-memory and message-passing approaches.

1. Definition

Inter-Process Communication (IPC) is a mechanism that allows two or more processes to exchange information and coordinate their activities.

IPC is important because processes often need to share data or communicate while executing concurrently.

2. Need for IPC

IPC is used for:

1. Information sharing.
2. Data exchange.
3. Process synchronization.
4. Coordination of concurrent processes.
5. Modular application development.
6. Communication between processes on different computers.

3. Types of IPC mechanisms

The two major IPC models are:

- Shared memory.
- Message passing.

Other mechanisms include pipes, message queues, sockets, and signals.

4. IPC architecture

🟦 INTER-PROCESS COMMUNICATION

Process P1

Process P2

IPC Mechanism

Shared Memory / Message Passing

Data Exchange and Coordination

5. Shared-memory approach

Shared memory is an IPC mechanism in which two or more processes access a common region of memory.

The OS establishes the shared-memory region, and processes communicate by reading and writing data in that region.

🟩 SHARED-MEMORY IPC

Process P1

Writes data

Process P2

Reads data

Shared Memory Region

Common data accessible to both processes

Processes access the shared region directly after it has been established.

Working steps:

1. The OS creates a shared-memory region.
2. The region is mapped into the address spaces of participating processes.
3. Process P1 writes data.
4. Process P2 reads the data.
5. Synchronization controls access when both processes may modify shared data.

Example: Two processes share a buffer for exchanging data.

Advantages:

- Fast data exchange.
- Suitable for large amounts of data.
- Less copying after the shared region is established.
- Efficient for processes on the same computer.

Disadvantages:

- Requires synchronization.
- Incorrect access can cause race conditions.
- Shared data must be carefully managed.
- Usually intended for processes on the same system.

6. Message-passing approach

Message passing is an IPC mechanism in which processes exchange information by sending and receiving messages.

The OS or communication system provides operations to send and receive data.

MESSAGE-PASSING IPC

Process P1

Send message

Message Queue / IPC System

Receive message

Process P2

Working steps:

1. Process P1 prepares a message.
2. P1 calls a send operation.
3. The OS or IPC mechanism transfers or queues the message.
4. Process P2 calls a receive operation.
5. P2 receives and processes the message.

Examples:

- Pipes.
- Message queues.
- Sockets.
- Mailboxes.

Advantages:

- Easier communication structure.
- No common memory region required.
- Synchronization may be integrated into the communication mechanism.
- Can support communication across computers.

Disadvantages:

- Message transfer may add overhead.
- Data may need to be copied.
- Large messages can be expensive.

- Communication setup may be more complex.

7. Shared memory vs message passing

Feature	Shared memory	Message passing
Communication	Common memory region	Send and receive messages
Data access	Direct access after setup	Through communication operations
Speed	Usually faster for large local data	May have transfer overhead
Synchronization	Often explicitly required	May be built into IPC mechanism
Data copying	Can be reduced	May involve copying
System scope	Usually same computer	Same or different computers
Complexity	Shared-data coordination	Message protocol coordination
Example	Shared buffer	Pipe or socket

8. Example: Producer-consumer problem

A producer generates data, while a consumer processes it.

- Producer places data in a buffer.
- Consumer removes data from the buffer.
- Both processes coordinate to avoid accessing the buffer incorrectly.

In shared memory, they access a common buffer.

In message passing, the producer sends messages and the consumer receives them.

Conclusion: IPC enables processes to communicate and coordinate. Shared memory is efficient for local data sharing, while message passing provides structured communication and can support distributed systems.

Q15. Analyze how process scheduling and inter-process communication work together to support concurrent process execution in an operating system.

1. Definition

Concurrent execution is the execution of multiple processes during overlapping periods of time.

On a single CPU, processes may take turns using the processor. On multicore systems, multiple processes or threads can execute simultaneously.

Process scheduling determines which process receives the CPU.

Inter-process communication (IPC) allows processes to exchange data and coordinate their activities.

Together, scheduling and IPC support effective concurrent execution.

2. Relationship between scheduling and IPC

🟦 CONCURRENT PROCESS EXECUTION

Process P1

Process P2

Ready Queue

CPU Scheduler

CPU Execution

IPC / Synchronization

Exchange data and coordinate processes

Scheduling allocates CPU time; IPC allows concurrent processes to exchange information.

3. Role of process scheduling

Process scheduling:

1. Selects a process from the ready queue.
2. Allocates CPU time.
3. Supports process switching.
4. Allows multiple processes to make progress.
5. Provides fairness and responsiveness.
6. Handles processes that block or become ready.

Example: P1 executes first, then the scheduler selects P2, then later resumes P1.

4. Role of IPC

IPC allows concurrent processes to:

- Exchange information.
- Share data.
- Coordinate execution.
- Notify one another about events.

- Synchronize access to resources.

Examples include shared memory, pipes, message queues, and sockets.

5. How they work together

Consider a producer and consumer process.

Step 1: Process creation

The OS creates the producer and consumer processes.

Step 2: Ready queue

Both processes enter the ready queue when they are eligible to execute.

Step 3: Scheduling

The CPU scheduler selects the producer.

Step 4: Data production

The producer generates data and places it into a shared buffer or sends a message.

Step 5: Synchronization

If the buffer is full, the producer may wait. If the buffer is empty, the consumer may wait.

Step 6: Process switching

The scheduler selects another ready process while one process waits.

Step 7: Data consumption

The consumer receives data and processes it.

Step 8: Resume

A blocked process becomes ready when the required event occurs.

6. Producer-consumer diagram

🟩 PRODUCER–CONSUMER MODEL

Producer Process

Generates data

Shared Buffer / Message Queue

Consumer Process

Receives and processes data

Scheduler + Synchronization

7. Benefits of combining scheduling and IPC

A. Improved CPU utilization

When one process waits for I/O or communication, another process can use the CPU.

B. Better responsiveness

Interactive processes can receive CPU time while background processes wait.

C. Efficient resource sharing

IPC allows processes to exchange data instead of duplicating all information.

D. Coordination

Synchronization ensures that processes operate in the correct order when required.

E. Concurrent problem solving

A complex task can be divided among cooperating processes.

8. Challenges

- Race conditions.
- Deadlocks.
- Synchronization overhead.
- Context-switching overhead.
- Communication delays.
- Resource contention.

9. Example of concurrent execution

Suppose three processes are running:

Process	Activity
P1	Produces data
P2	Processes data
P3	Performs file I/O

The scheduler allocates CPU time among them.

When P1 sends data to P2, IPC transfers the data.

When P3 waits for file input, the scheduler can execute P1 or P2.

Thus, scheduling and IPC together allow the processes to make progress without requiring each process to execute continuously.

Conclusion: Process scheduling manages CPU allocation, while IPC enables data exchange and coordination. Together, they support concurrent execution, improve resource utilization, and provide responsive and efficient operating-system operation.